实验一: OpenEuler 内核编译与替换 实验报告

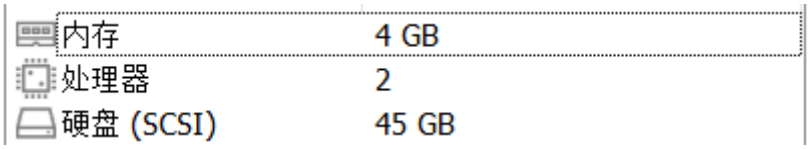
刘晨阳520030910301
2022.10.3

一.实验目的与要求

在 Linux 系统上，下载 openEuler 内核进行编译与替换，进而熟悉 Linux 的开发环境，掌握 Linux 内核编译的过程，了解 openEuler 内核，能够编译替换不同版本的内核，为后续自行修改内核代码并编译替换铺垫。

二.实验环境搭建

本次实验选择了**VMware Workstation 16 +Ubuntu18.04**的虚拟环境来进行内核替换，虚拟机配置如下图所示：

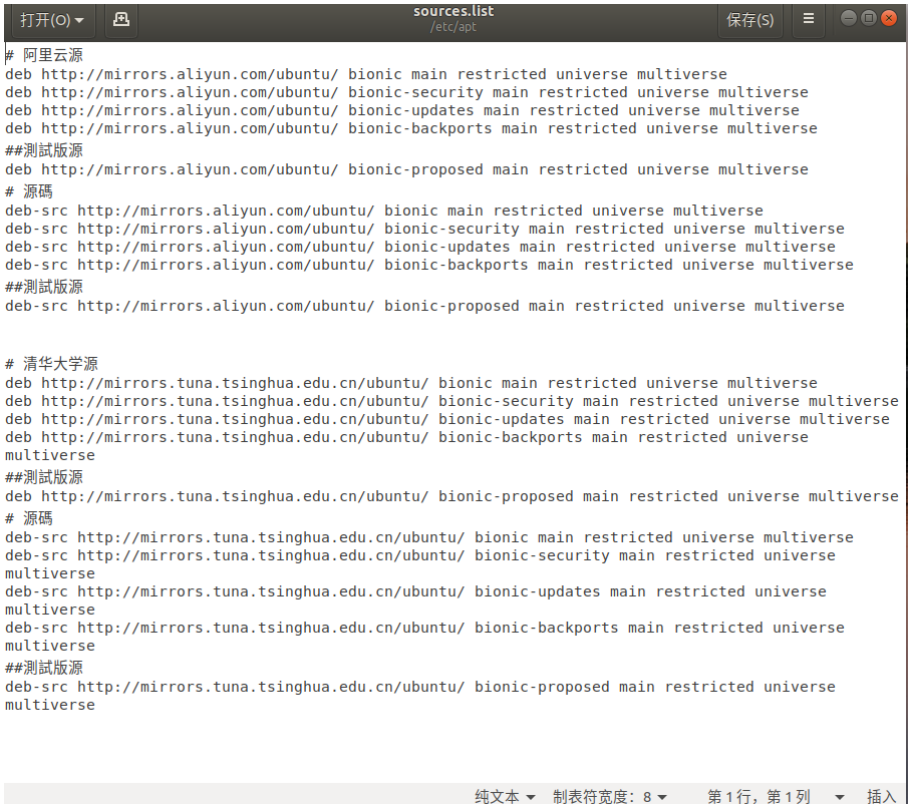


由于内核编译过程需要 20G 以上存储空间，虚拟机硬盘需要分配 40G 以上，并且为了加快编译，提高性能，内存和处理器也做了相应的资源提升。

并且，本实验需要众多的依赖包，由于以前出现过因为依赖缺失的各种各样的问题，这里首先进行换源。终端输入如下指令，先备份再换源：

```
sudo cp /etc/apt/sources.list /etc/apt/sources.list.backup

sudo gedit /etc/apt/sources.list
```

将打开的文档换为阿里和清华源即可



三.实验操作

I.内核代码下载

在gitee上下载OpenEuler内核源码，链接为https://gitee.com/openeuler/kernel/releases。这里我选择了OpenEuler的**4.19.90**-2209.6.0版本

II.内核编译及安装

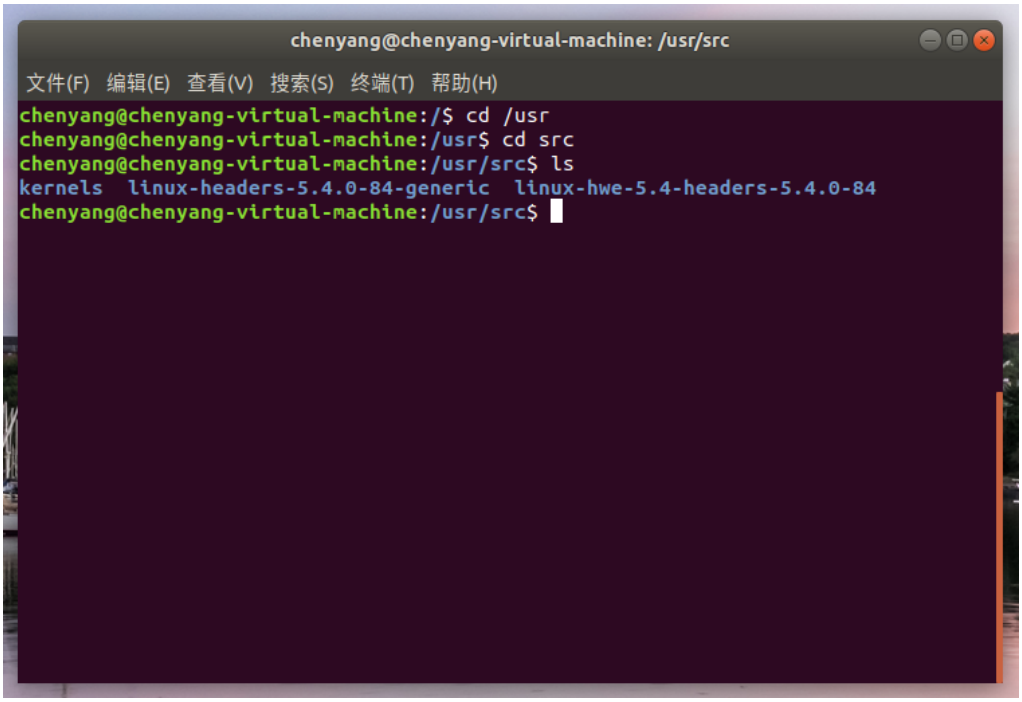
1.解压缩

将下载好的源码进行解压，在压缩包所在目录输入如下指令：

```
tar xvf kernel-4.19.90-2209.6.0.tar.gz
```

会得到一个同名的解压好的内核源码文件夹，我将文件夹名修改为内核版本的名称然后将它移动到内核源码目录下方便统一管理各版本的源码，我将kernel文件以及我的其他几个linux内核文件都放在*/usr/src/kernels*。接下来的大部分内核编译操作都在*kernels*这个文件夹里进行。

```
mv kernel-4.19.90-2209.6.0 4.19.90
mv 4.19.90/ /usr/src/kernels/
```



2.清理源代码树

进入/*kernels*执行命令，清理过去内核编译产生的文件，第一次编译时可不执行此命令，但之后每一次编译失败后重新来过都需清理。

```
make mrproper
```

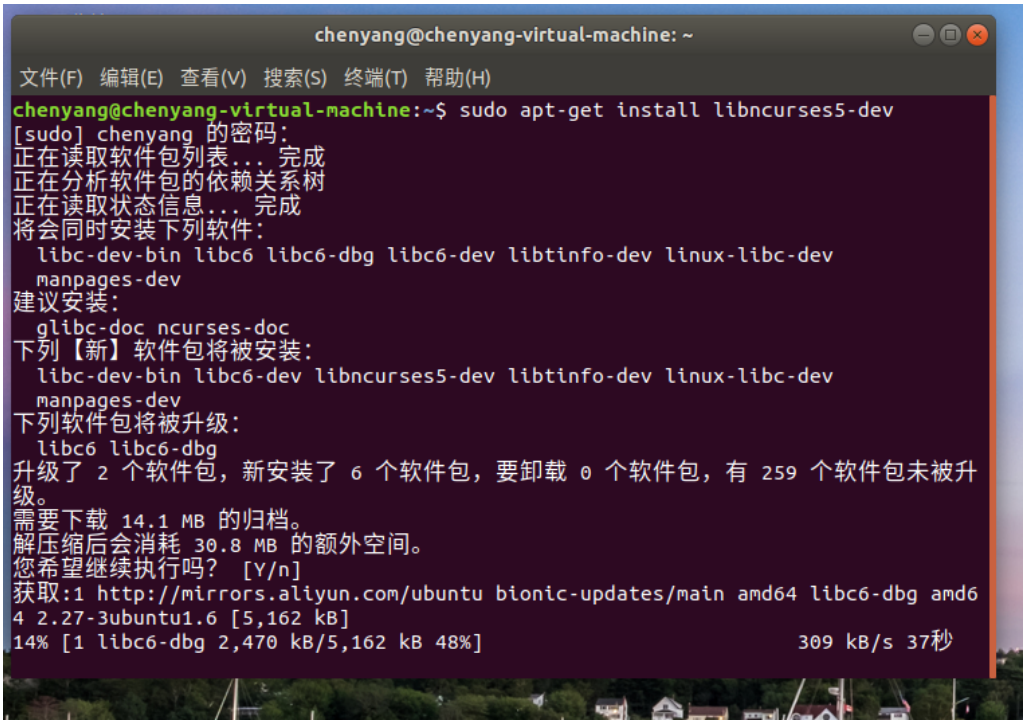
3.内核配置

首先需要生成内核配置文件，本实验使用系统原配置的默认选项，直接将系统原配置文件拷贝过来，原配置文件在/*boot*目录下。

```
cp -v /boot/config-$(uname -r) .config
```

接着，需要安装一系列依赖，才可以对内核配置进行更改。由于本实验的虚拟机之前从未使用过，缺失了很多关键依赖，如`flex,gcc,libncurses5-dev`等，这些包在换源后可以快速安装。

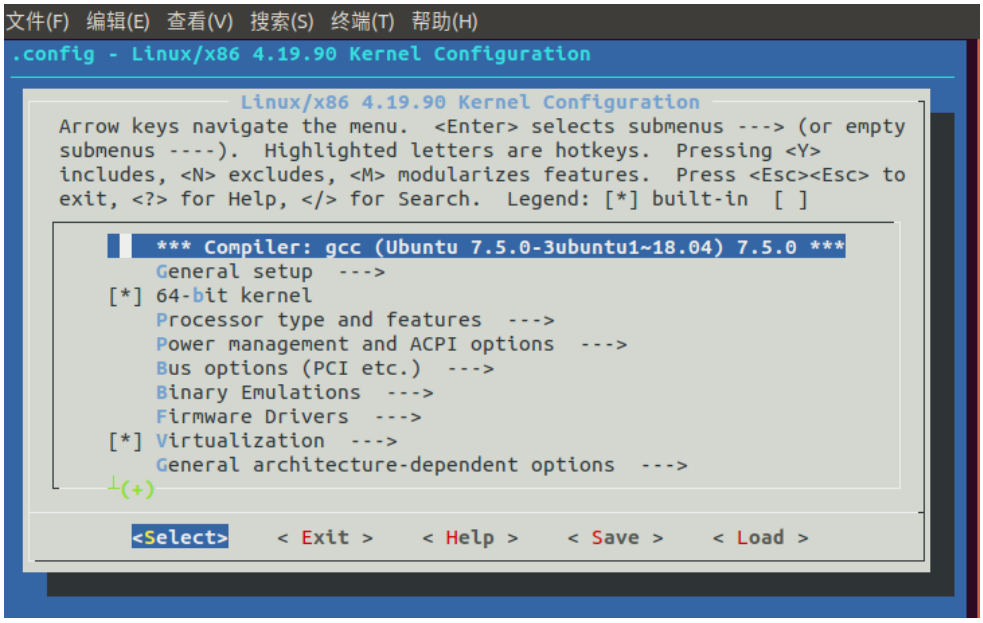
```
sudo apt-get update

sudo apt-get install flex gcc libncurses5-dev
```



然后对配置进行需要的更改，决定将内核的各个功能系统编译进内核还是编译为模块还是不编译。在内核目录输入如下命令：

```
make menuconfig
```

如果上述依赖安装正确，会在命令执行后会进入基于文本选单的配置界面：



先Load载入原始*.config*配置，本实验没有改动，直接默认配置，然后选择Save，生成了一个*.config*文件。

4.内核编译及安装

内核编译需要一系列依赖，主要有`build-essential,libelf-dev,openssl,libssl-dev,bc` 等包,一个命令即可完成：

```
sudo apt install -y build-essential flex bison libssl-dev libelf-dev openssl
```

然后，虽然根据实验指导下一步可以进行编译了，但我在实验中编译了好几次都中断了，总是出现`make[1]: *** 没有规则可制作目标“debian/canonical-certs.pem”，由“certs/x509_certificate_list”`究其原因是*.config*文件的问题，我修改了相关*.config*文件才编译成功，即将图中高亮部分的CONFIG_SYSTEM_TRUSTER_KEYS右边的引号里的内容删除。

```
#
# Certificates for signature checking
#
CONFIG_MODULE_SIG_KEY="certs/signing_key.pem"
CONFIG_SYSTEM_TRUSTED_KEYRING=y
CONFIG_SYSTEM_TRUSTED_KEYS=""
CONFIG_SYSTEM_EXTRA_CERTIFICATE=y
CONFIG_SYSTEM_EXTRA_CERTIFICATE_SIZE=4096
CONFIG_SECONDARY_TRUSTED_KEYRING=y
CONFIG_SYSTEM_BLACKLIST_KEYRING=y
CONFIG_SYSTEM_BLACKLIST_HASH_LIST=""
```

删去引号内原有的内容

接下来就可以编译了，为了加快速度，我使用了`make -j8`，在双核CPU上并行执行编译，经过大概1个小时编译就成功结束了。下图是编译结束的日志截图：

```
chenyang@chenyang-virtual-machine: /usr/src/kernels
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
  LD [M]  sound/soc/snd-soc-acpi.ko
  LD [M]  sound/soc/snd-soc-core.ko
  LD [M]  sound/soc/xtensa/snd-soc-xtfpga-i2s.ko
  LD [M]  sound/soc/zte/zx-tdm.ko
  LD [M]  sound/synth/emux/snd-emux-synth.ko
  LD [M]  sound/synth/snd-util-mem.ko
  LD [M]  sound/soundcore.ko
  LD [M]  sound/usb/6fire/snd-usb-6fire.ko
  LD [M]  sound/usb/bcd2000/snd-bcd2000.ko
  LD [M]  sound/usb/caiaq/snd-usb-caiaq.ko
  LD [M]  sound/usb/hiface/snd-usb-hiface.ko
  LD [M]  sound/usb/line6/snd-usb-pod.ko
  LD [M]  sound/usb/line6/snd-usb-podhd.ko
  LD [M]  sound/usb/line6/snd-usb-line6.ko
  LD [M]  sound/usb/line6/snd-usb-toneport.ko
  LD [M]  sound/usb/line6/snd-usb-variax.ko
  LD [M]  sound/usb/misc/snd-ua101.ko
  LD [M]  sound/usb/snd-usb-audio.ko
  LD [M]  sound/usb/snd-usbmidi-lib.ko
  LD [M]  sound/usb/usx2y/snd-usb-usx2y.ko
  LD [M]  sound/x86/snd-hdmi-lpe-audio.ko
  LD [M]  sound/usb/usx2y/snd-usb-us122l.ko
  LD [M]  sound/xen/snd_xen_front.ko
chenyang@chenyang-virtual-machine:/usr/src/kernels$
```

编译完成后安装模块

```
make modules_install
```

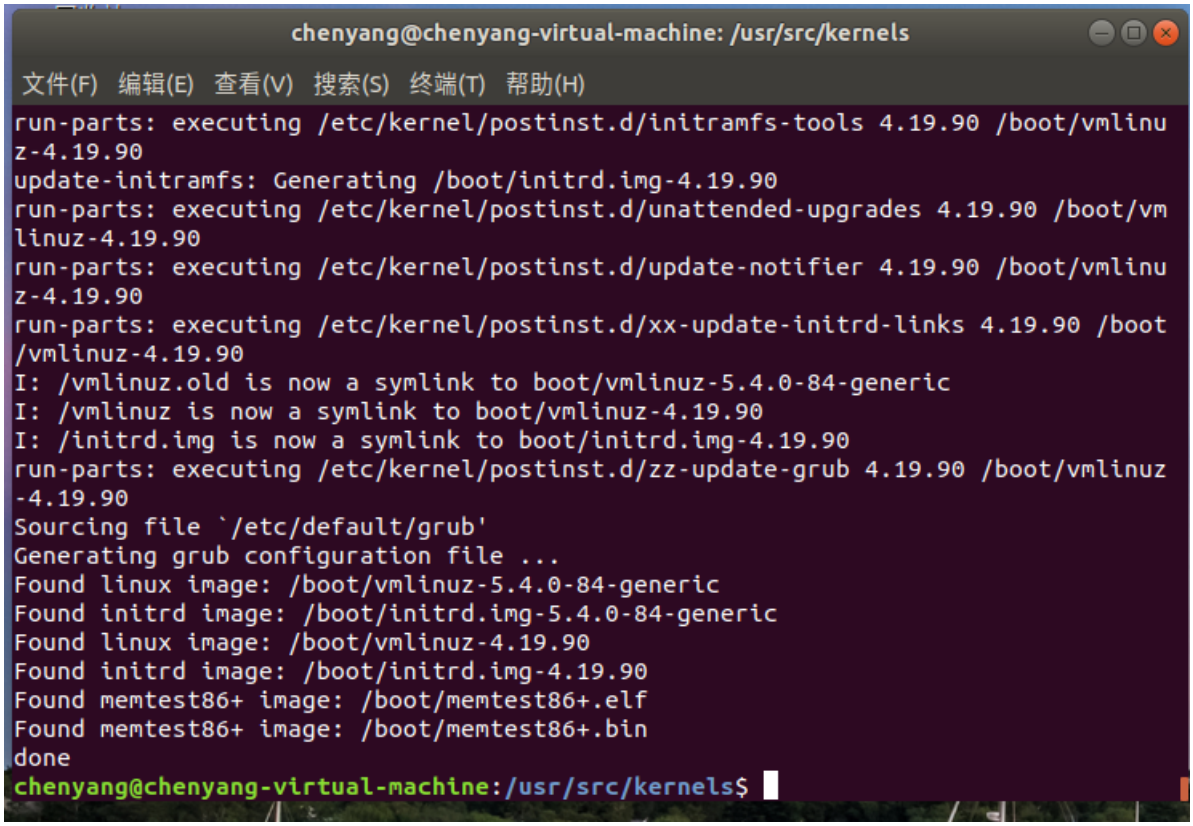
以下是安装模块结束后的截图：

```
chenyang@chenyang-virtual-machine: /usr/src/kernels
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
  INSTALL sound/soc/snd-soc-core.ko
  INSTALL sound/soc/xtensa/snd-soc-xtfpga-i2s.ko
  INSTALL sound/soc/zte/zx-tdm.ko
  INSTALL sound/soundcore.ko
  INSTALL sound/synth/emux/snd-emux-synth.ko
  INSTALL sound/synth/snd-util-mem.ko
  INSTALL sound/usb/6fire/snd-usb-6fire.ko
  INSTALL sound/usb/bcd2000/snd-bcd2000.ko
  INSTALL sound/usb/caiaq/snd-usb-caiaq.ko
  INSTALL sound/usb/hiface/snd-usb-hiface.ko
  INSTALL sound/usb/line6/snd-usb-line6.ko
  INSTALL sound/usb/line6/snd-usb-pod.ko
  INSTALL sound/usb/line6/snd-usb-podhd.ko
  INSTALL sound/usb/line6/snd-usb-toneport.ko
  INSTALL sound/usb/line6/snd-usb-variax.ko
  INSTALL sound/usb/misc/snd-ua101.ko
  INSTALL sound/usb/snd-usb-audio.ko
  INSTALL sound/usb/snd-usbmidi-lib.ko
  INSTALL sound/usb/usx2y/snd-usb-us122l.ko
  INSTALL sound/usb/usx2y/snd-usb-usx2y.ko
  INSTALL sound/x86/snd-hdmi-lpe-audio.ko
  INSTALL sound/xen/snd_xen_front.ko
  DEPMOD  4.19.90
chenyang@chenyang-virtual-machine:/usr/src/kernels$
```

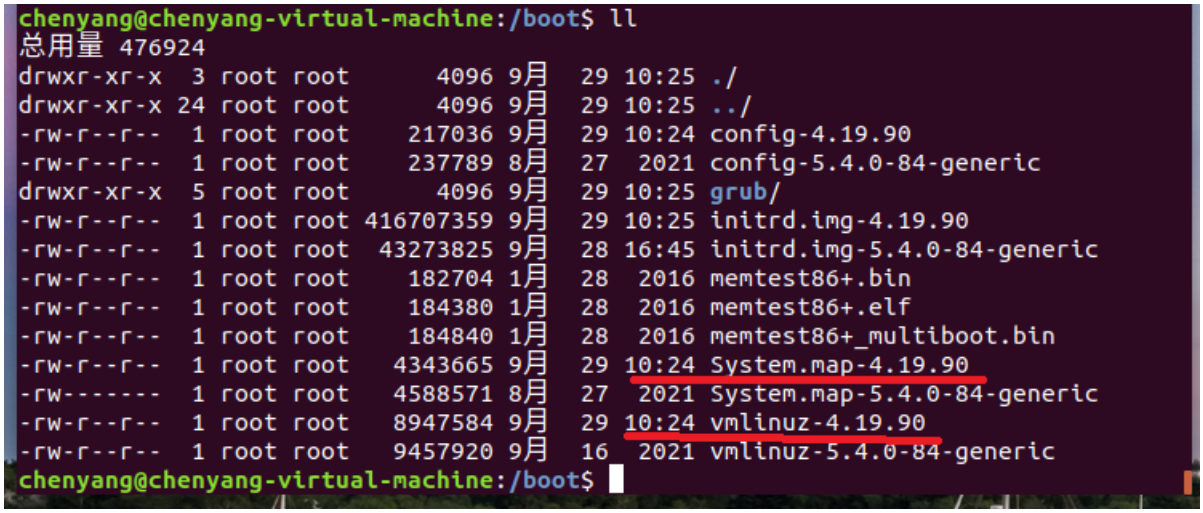
然后安装内核

```
make install
```

内核安装结束后如下图：



此时可以在/*boot*目录下看到新安装的内核：（如下图红线标出所示）



5.更新引导

首先使用下面的命令会根据 /*boot* 目录下的内核文件自动更新启动引导文件

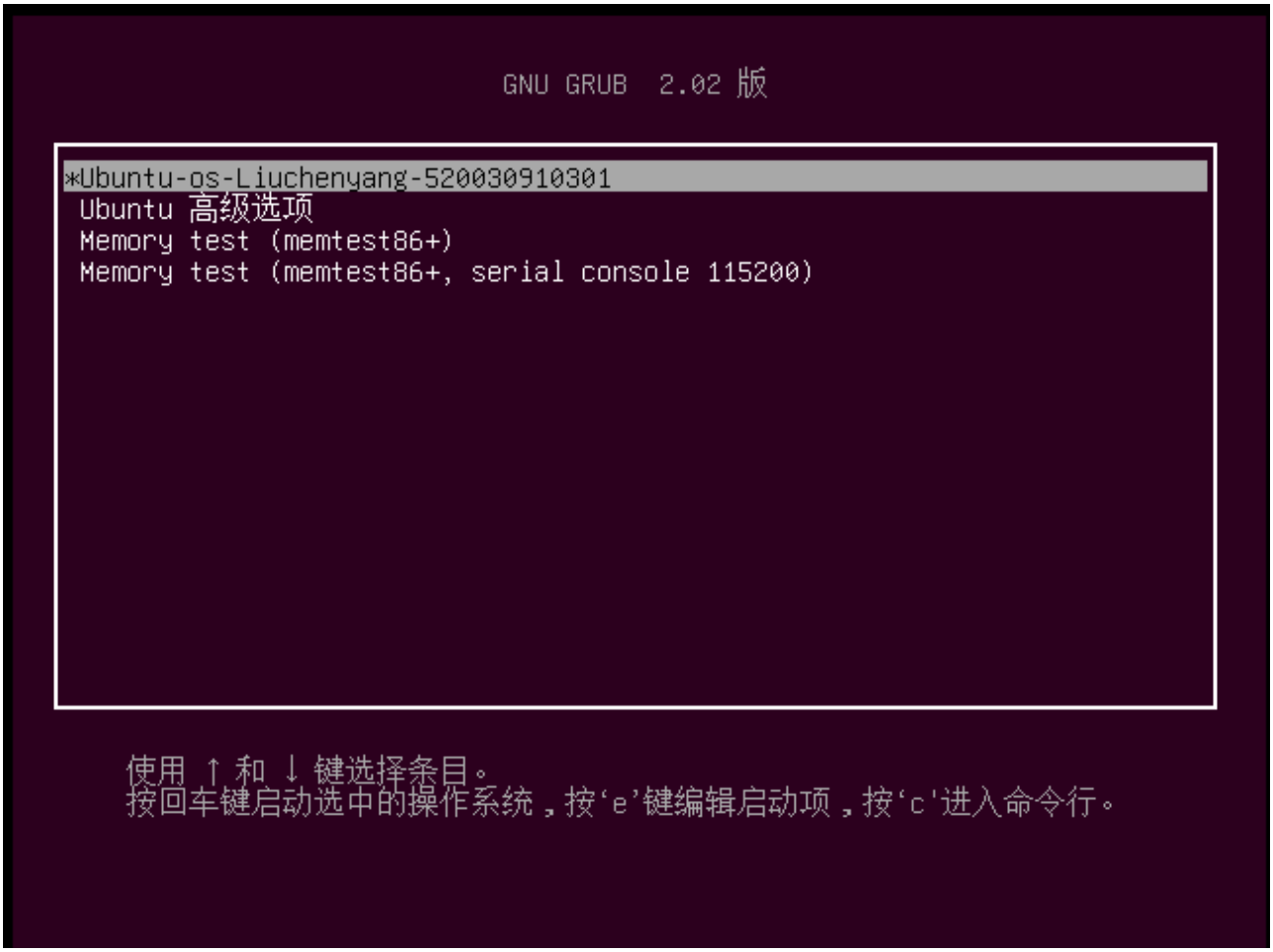
```
grub-mkconfig -o /boot/grub/grub.cfg
```

或者使用`sudo upgrade-grub`，都可以更新启动引导文件。

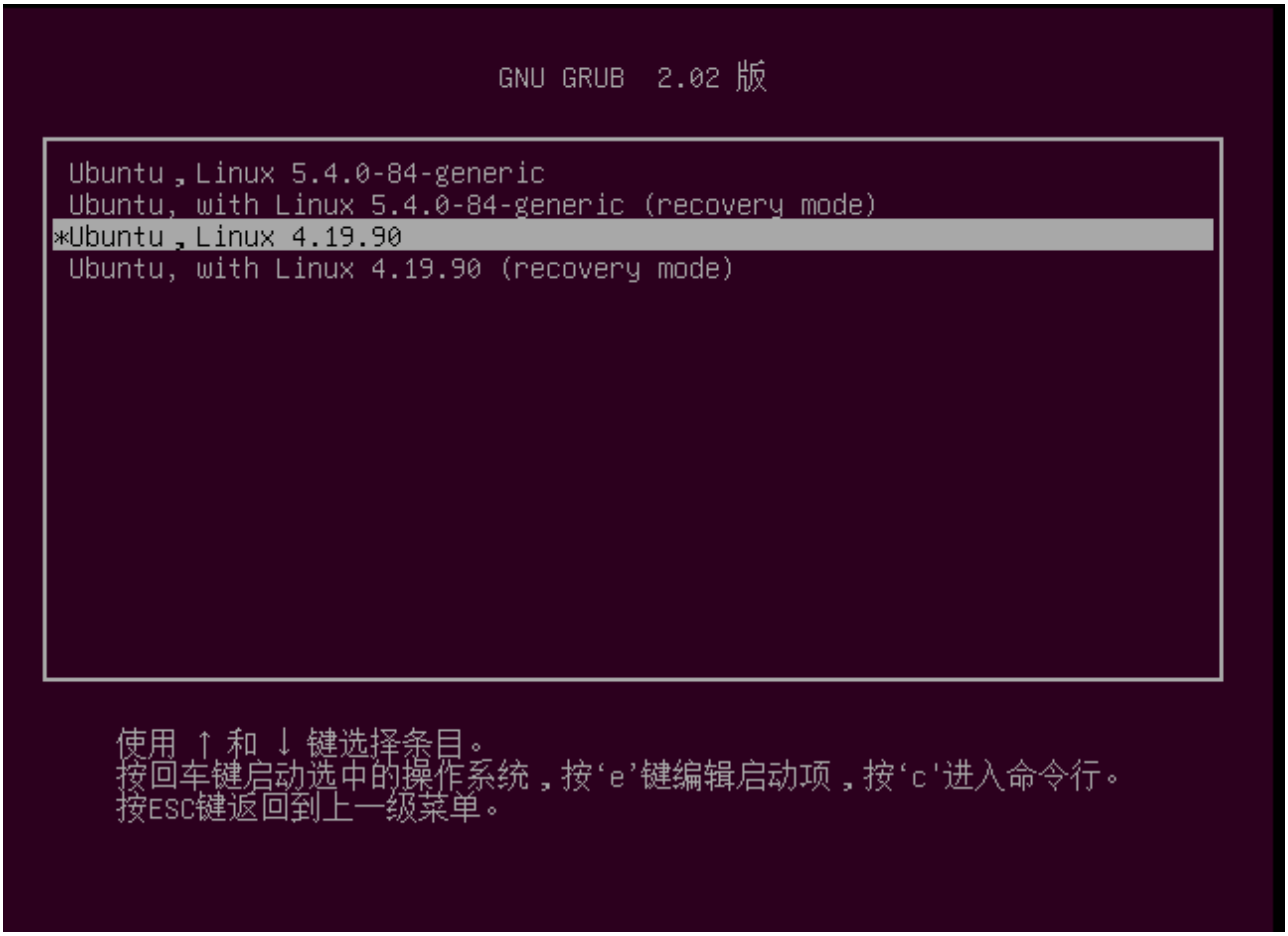
接下来就可以进入cfg文件 menuentry 后面的字符串，增加我的姓名与学号，自定义启动菜单时的选项名称，如下图红线所示：

```
menuentry 'Ubuntu-os-Liuchenyang-520030910301' --class ubuntu --class gnu-linux --class gnu --
class os $menuentry_id_option 'gnulinux-simple-97b46642-2adc-475c-a9cb-ece5841d2db9' {
        recordfail
        load_video
        gfxmode $linux_gfx_mode
        insmod gzio
        if [ x$grub_platform = xxen ]; then insmod xzio; insmod lzopio; fi
        insmod part_msdos
        insmod ext2
        set root='hd0,msdos1'
        if [ x$feature_platform_search_hint = xy ]; then
          search --no-floppy --fs-uuid --set=root --hint-bios=hd0,msdos1 --hint-efi=hd0,msdos1 --
hint-baremetal=ahci0,msdos1  97b46642-2adc-475c-a9cb-ece5841d2db9
        else
          search --no-floppy --fs-uuid --set=root 97b46642-2adc-475c-a9cb-ece5841d2db9
        fi
        linux   /boot/vmlinuz-5.4.0-84-generic root=UUID=97b46642-2adc-475c-a9cb-ece5841d2db9 ro
quiet splash $vt_handoff
        initrd  /boot/initrd.img-5.4.0-84-generic
}
submenu 'Ubuntu 高级选项' $menuentry_id_option 'gnulinux-advanced-97b46642-2adc-475c-a9cb-
ece5841d2db9' {
```

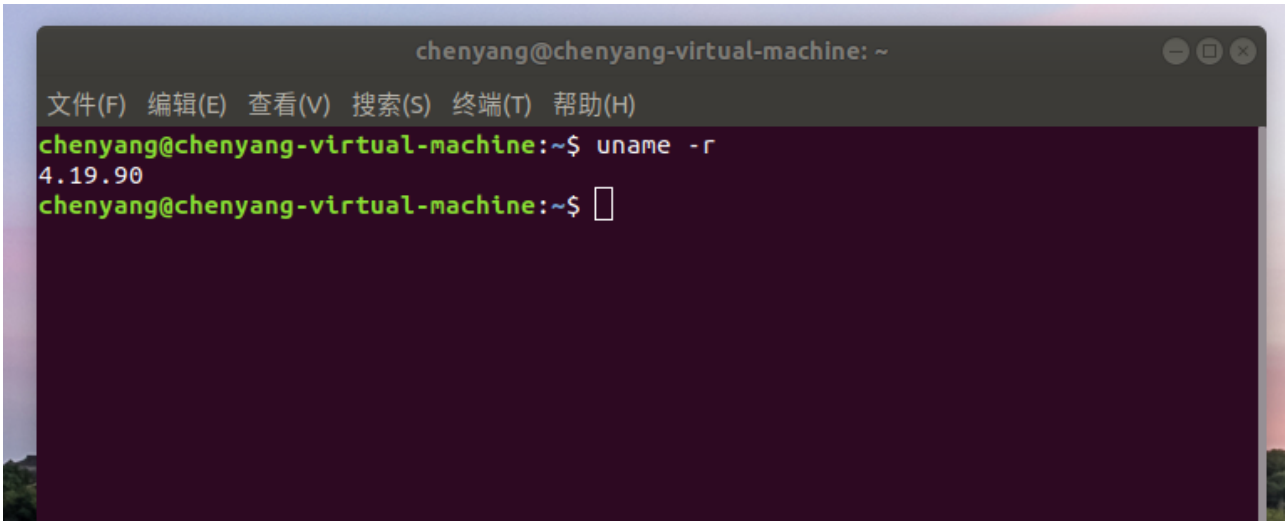
然后重启虚拟机，在刚刚启动时按住**ESC**，就会进入到内核选择可视化界面：



选择**Ubuntu**高级选项，看到如下界面：



选择版本即可使用该内核启动系统，开机后可以通过如下方式验证内核版本号



至此本实验操作结束，**OpenEuler-20.03-4.19.90**内核替换成功！

四.实验过程中遇到的错误及解决

这是我第一次更换操作系统内核，实验过程中踩了很多坑，但最终都一一解决。

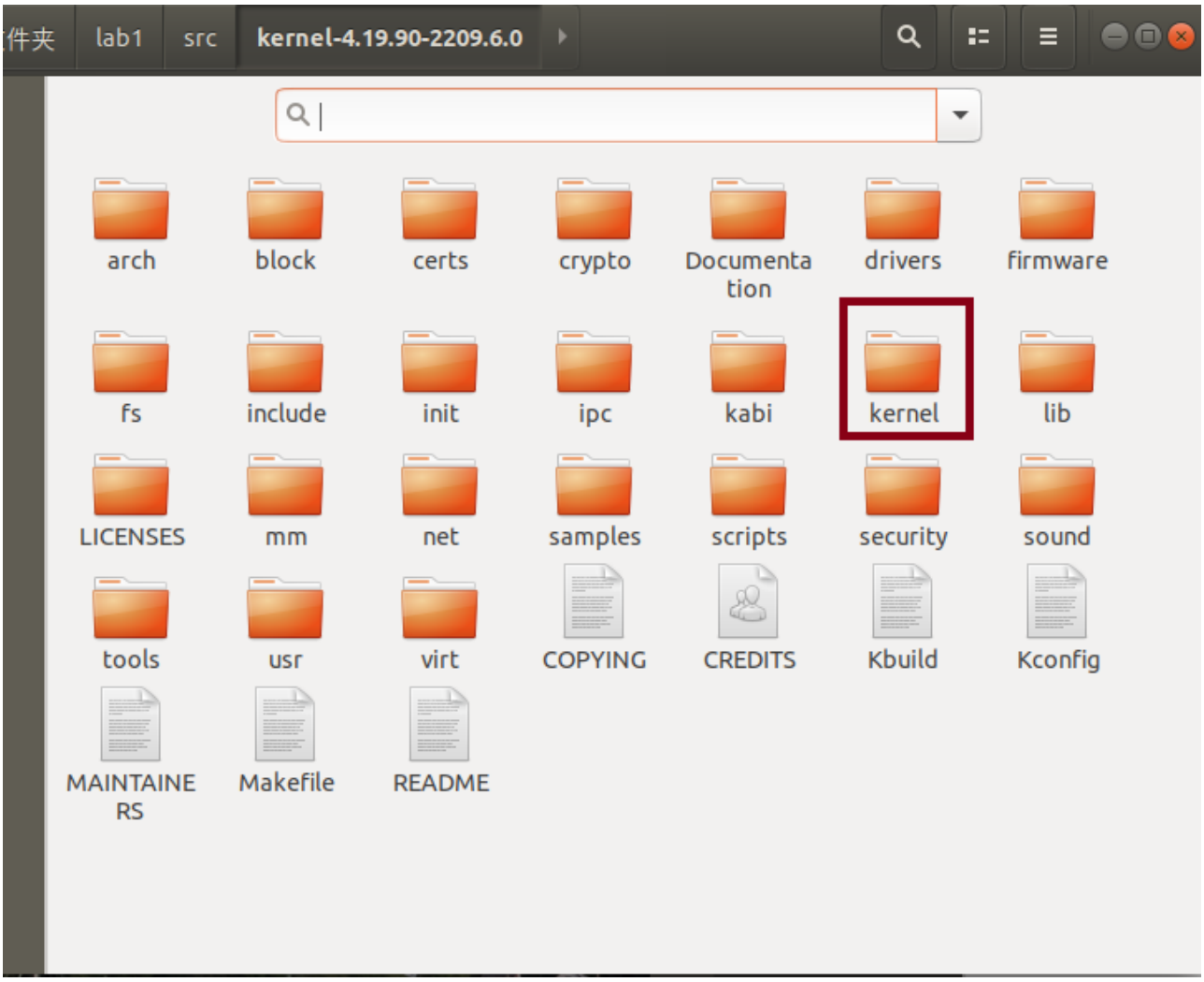
1.各种依赖包缺失

实验过程中，由于我的Ubuntu是新创建的虚拟机，中间无论是编译内核还是安装内核，都报了很多莫名其妙的错误。比如在内核配置的时候，报错重要的包`libncurses5-dev`缺失；在编译时候报错gcc缺失，flex缺失。但如果换好了国内的源，这些都可以通过`apt-get`安装成功。

2.内核配置没有在内核文件夹中执行

这其实是一个非常粗心的错误，我误以为内核文件中的*kernel*文件是所谓的内核文件，在这个子文件中无论怎么努力都无法使得`make menuconfig`成功，总是报错：`make: *** 没有规则可以创建目标“menuconfig”。 停止`。在分析之后，我发现所有的依赖包都已经安装完毕，我又尝试在Ubuntu自带的源码文件中验证了一下，才发现是需要在kernel上一级，也就是解压的得到的那一级文件执行。这让我了解到了linux文件系统的精妙严谨之处。

如下图，内核文件夹里的*kernel*其实包含了很多内核源代码，但不能在其内部进行内核配置，要在实际的*kernel-4.19.90-2209.6.0*文件中进行。



3..config文件需要更改才可编译

这个错误其实非常难以理解，因为实验文档中基本忽略了*.config*文件的配置。但在本次实验中，我尝试了前3次编译都在途中突然停止并异常退出，报错：`make[1]: *** 没有规则可制作目标“debian/canonical-certs.pem”，由“certs/x509_certificate_list”错误`，然而我多次检查后并没有发现我有缺失的依赖包和错误的操作，这让我十分困惑。

经过查询资料，我发现解决方案就是将*.config*文件中的一行内容删去，然后就可顺利通过编译。

```
#
# Certificates for signature checking
#
CONFIG_MODULE_SIG_KEY="certs/signing_key.pem"
CONFIG_SYSTEM_TRUSTED_KEYRING=y
CONFIG_SYSTEM_TRUSTED_KEYS=""
CONFIG_SYSTEM_EXTRA_CERTIFICATE=y
CONFIG_SYSTEM_EXTRA_CERTIFICATE_SIZE=4096
CONFIG_SECONDARY_TRUSTED_KEYRING=y
CONFIG_SYSTEM_BLACKLIST_KEYRING=y
CONFIG_SYSTEM_BLACKLIST_HASH_LIST=""
```

删去引号内原有的内容

4.关于grub和grub2

我在我的终端上进行更新引导时，输入`grub2-mkconfig -o /boot/grub2/grub.cfg`的时候会报错，找不到`/boot/grub2/grub.cfg`，我发现我的grub和grub2的包也是安装并且最新的。查阅资料后发现，有的系统可能不支持grub2。实际上，grub2是版本号在1.98之后的grub；grub一般指grub legacy，版本号为0.97。二者在本次实验中用哪个其实并不影响，于是，我改用输入下面这行即可：

```
grub-mkconfig -o /boot/grub/grub.cfg
```

但后来，我发现输入如下指令更为简便：也可以更新引导文件。

```
sudo update-grub
```

```
chenyang@chenyang-virtual-machine:~$ sudo update-grub
[sudo] chenyang 的密码：
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-5.4.0-84-generic
Found initrd image: /boot/initrd.img-5.4.0-84-generic
Found linux image: /boot/vmlinuz-4.19.90
Found initrd image: /boot/initrd.img-4.19.90
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
done
chenyang@chenyang-virtual-machine:~$
```

五.实验总结

本次实验中我顺利更换了OpenEuler的内核，虽然遇到很多困难，但都可以通过自己查找资料一一解决。正所谓苦尽甘来，当搜索资料有了结果的时候，当电脑风扇呼啸1个小时后停下结束编译的时候，当最终更换的内核版本可以成功显示的时候，我的内心是喜悦无比的，我也学到了不少关于linux命令行，环境配置，操作系统内核的知识，这为我接下来实验与学习操作系统打下了基础，进一步激发了我对操作系统等底层系统软件的兴趣。